

rest-sample-cache-reader Kullanıcı Rehberi

Kısa kullanıcı rehberi

Bu rehber ilk kullanım içindir.

Amaç kısa ve nettir: Redis içinde hazır duran JSON verisini REST API olarak döndürmek.

İçindekiler

1. Bu Proje Ne İşe Yarar?
2. Akış Nasıl Çalışır?
3. Ne Zaman Kullanılır?
4. Gerçek Hayat Senaryoları
5. Hızlı Başlangıç
6. Kopyala-Yapıştır Örnekler
7. Endpoint'ler
8. Önemli Ayarlar
9. Tuning Karar Tablosu
10. Production Reçeteleri
11. Sık Hatalar

Bu Proje Ne İşe Yarar?

rest-sample-cache-reader, düşük memory hedefli bir REST cache reader örneğidir.

Uygulama DB'ye gitmez. Scheduler çalıştırmaz. Dubbo kullanmaz.

Sadece Redis'ten okur ve HTTP JSON response döner.

Akış Nasıl Çalışır?

Akış diyagramı:

flowchart LR

```
A["Client"] --> B["Rust-Java REST API"]
B --> C["Java Handler"]
C --> D["java-rust-cache"]
D --> E["Rust Redis Client"]
E --> F["Redis Snapshot"]
F --> E --> D --> C --> B --> A
```

Java tarafı business route kararını verir.

Redis I/O Rust tarafındadır. Bu yüzden Java Redis client ve ekstra Netty yüzeyi yoktur.

Ne Zaman Kullanılır?

Senaryo	Bu proje uygun mu?	Neden
Redis'te hazır JSON var	Evet	En düşük maliyetli read path budur.
API DB'ye anlık query atacak	Hayır	Bu proje DB bağlantısı açmaz.
Çok düşük RSS isteniyor	Evet	micro-rest ve küçük Redis pool ile çalışır.
Cache yoksa DB fallback isteniyor	Hayır	Fallback başka servis veya writer sorumluluğudur.
Read-heavy lookup API	Evet	Hazır snapshot okur, DTO graph kurmaz.

Gerçek Hayat Senaryoları

Senaryo	Ne olur?	Nasıl kullanılır?
CRM müşteri kartı	Kullanıcı müşteri detay ekranı açar. API Redis'teki hazır customer JSON'u döner.	<code>GET /api/v1/cache/customers/{id}</code>
Kampanya ekranı	Kampanya motoru aktif müşteri adaylarını hızlı ister. DB'ye gidilmez.	<code>GET /api/v1/cache/customers/campaigns/retention/candidates</code>
Segment dashboard	Operasyon ekibi segment bazlı müşteri listesini görür.	<code>GET /api/v1/cache/customers/segments/standard</code>
Cache hazır değil	Writer henüz snapshot yazmadıysa reader 404 döner.	Caller kısa süre sonra tekrar dener.
Redis HA	Production ortamında Sentinel veya Cluster kullanılır.	<code>reactor.cache.redis.topology</code> değeri değiştirilir.

Bu projede en önemli karar şudur: API hızlı olsun diye veriyi önceden Redis'e yazıyoruz.

Request geldiğinde DB query çalıştırmıyoruz.

Hızlı Başlangıç

Redis'i başlatın:

```
docker run --name sample-redis -p 6379:6379 -d redis:7-alpine
```

Uygulamayı Maven ile çalıştırın:

```
mvn -q package
java -jar target/rest-sample-cache-reader-0.1.0.jar
```

Health kontrolü:

```
curl http://127.0.0.1:8080/app/health
```

Cache hazır değilse customer endpoint'leri 404 dönebilir. Bu normaldir. Önce writer uygulaması Redis'e snapshot yazmalıdır.

Kopyala-Yapıştır Örnekler

Customer ID ile oku:

```
curl http://127.0.0.1:8080/api/v1/cache/customers/1
```

Customer numarası ile oku:

```
curl "http://127.0.0.1:8080/api/v1/cache/customers/by-customer-no?customerNo=CUST-1001"
```

Segment listesi oku:

```
curl http://127.0.0.1:8080/api/v1/cache/customers/segments/standard
```

Status listesi oku:

```
curl http://127.0.0.1:8080/api/v1/cache/customers/statuses/active
```

Kampanya adaylarını oku:

```
curl http://127.0.0.1:8080/api/v1/cache/customers/campaigns/retention/candidates
```

Cache meta bilgisini kontrol et:

```
curl http://127.0.0.1:8080/api/v1/cache/customers/meta
```

Redis metriklerini kontrol et:

```
curl http://127.0.0.1:8080/api/v1/cache/customers/cache-metrics
```

Endpoint'ler

Endpoint	Ne döner?	Ne zaman kullanılır?
GET /app/health	Uygulama sağlık bilgisi	Pod health check
GET /api/v1/cache/customers/{id}	Customer detail JSON	ID ile hızlı okuma
GET /api/v1/cache/customers/by-customer-no?customerNo=CUST-1	Customer detail JSON	Business key ile okuma
GET /api/v1/cache/customers/segments/{segment}	Segment listesi	Liste veya dashboard
GET /api/v1/cache/customers/statuses/{status}	Status listesi	Aktif/pasif müşteri listesi
GET /api/v1/cache/customers/campaigns/{campaign}/candidates	Kampanya adayları	Read-heavy kampanya ekranı
GET /api/v1/cache/customers/meta	Snapshot meta bilgisi	Cache hazır mı kontrolü
GET /api/v1/cache/customers/cache-metrics	Cache metrikleri	Operasyonel kontrol

Örnek:

```
curl http://127.0.0.1:8080/api/v1/cache/customers/1
```

Önemli Ayarlar

Property	Ne işe yarar?	Ne zaman değiştirilir?
reactor.runtime.profile=micro-rest	REST runtime'ı düşük RSS için dar tutar.	Genelde değiştirmeyin.
sample.cache.customer.namespace=crm.customer	Redis key alanını belirler.	Aynı Redis'te birden fazla domain varsa değiştirin.
sample.cache.customer.projections=detail,segment,status,campaign,meta	Hangi projection'lar okunacak belirler.	Writer ile aynı olmalıdır.
sample.cache.customer.version-cache-ms=1000	Aktif snapshot version bilgisini kısa süre cache'ler.	Çok sık version değişiyorsa düşürün.
reactor.cache.redis.topology=standalone	Redis bağlantı tipidir.	Sentinel veya Cluster kullanıyorsanız değiştirin.
reactor.cache.redis.max-read-inflight=64	Aynı anda kaç Redis read uçuşta olabilir.	p99 ve RSS birlikte ölçülerek artırılır.
reactor.cache.redis.max-response-bytes=1048576	Tek Redis response üst limitidir.	Büyük JSON okuyorsanız artırın.

Tuning Karar Tablosu

Durum	İlk bakılacak ayar	Ne yapmalı?	Beklenen etki
RSS yüksek	reactor.cache.redis.max-read-inflight	Değeri düşürün.	Aynı anda daha az Redis response taşınır.
p99 yüksek	Redis latency ve max-read-inflight	Redis'i ölçün, in-flight değerini düşürün.	Kuyruk baskısı azalır.
Çok 404 cache_not_ready var	Writer durumu	Writer projection publish loglarını kontrol edin.	Snapshot hazır olunca 200 döner.
Büyük JSON dönüyor	reactor.cache.redis.max-response-bytes	Sadece gerekli endpoint için artırın.	Büyük response çalışır, RSS artabilir.
Sentinel kullanılıyor	reactor.cache.redis.topology=sentinel	Node listesi ve master adını verin.	Failover takip edilir.
Cluster kullanılıyor	reactor.cache.redis.topology=cluster	Cluster node listesini verin.	Key alanı cluster'a yayılır.

Production Reçeteleri

İhtiyaç	Başlangıç ayarı	Etki
En küçük pod	<code>micro-rest, max-read-inflight=32-64</code>	RSS düşük kalır, spike anında fail-fast olabilir.
Redis Sentinel	<code>reactor.cache.redis.topology=sentinel</code> ve <code>reactor.cache.redis.nodes=...</code>	Redis master değişimini takip eder.
Redis Cluster	<code>reactor.cache.redis.topology=cluster</code> ve <code>cluster node listesi</code>	Büyük key alanı için uygundur.
Büyük JSON	<code>max-response-bytes</code> artırılır	Memory bütçesini ayrıca ölçmek gerekir.

Sık Hatalar

Belirti	Muhtemel neden	Çözüm
<code>404 customer_cache_not_ready</code>	Writer henüz snapshot yazmadı.	Writer'ı çalıştırın ve <code>meta</code> endpoint'ini kontrol edin.
p99 yükseliyor	Redis yavaş veya in-flight fazla.	<code>max-read-inflight</code> değerini düşürün, Redis latency ölçün.
RSS büyüyor	Büyük JSON veya çok fazla concurrent response var.	Response boyutunu ve route concurrency değerlerini düşürün.
Cluster/Sentinel bağlanmıyor	Yanlış topology veya node listesi.	<code>reactor.cache.redis.topology</code> ve <code>reactor.cache.redis.nodes</code> değerlerini birlikte kontrol edin.